

Agregação com Junções em SQL Server

GROUP BY, HAVING e funções de agregação aplicados à malha fiscal

Curso de Banco de Dados Relacional aplicado à Fiscalização Tributária — SEFAZ-PB Banco de trabalho: `curso_integridade_fiscal` **SGBD:** Microsoft SQL Server 2016 ou superior (T-SQL)

Ambiente: SQL Server Management Studio (SSMS) ou VS Code + extensão MSSQL **Pré-requisito**

direto: módulo de Junções (JOINS)

Sumário

1. Objetivos de aprendizagem
 2. Do documento ao indicador: por que agregar
 3. A ordem lógica de processamento da consulta
 4. As funções de agregação
 5. Quadro resumo de todas as funções de agregação
 6. Agregação e NULL
 7. GROUP BY: a regra de ouro
 8. Agregação sobre junção interna
 9. Agregação sobre junção externa
 10. O fan-out: quando a junção falsifica o somatório
 11. HAVING: o filtro do grupo
 12. WHERE × HAVING × ON: os três filtros
 13. Subtotais: ROLLUP, CUBE e GROUPING SETS
 14. Agregação × funções de janela
 15. Indicadores fiscais construídos com agregação
 16. Erros comuns — quadro de diagnóstico
 17. Desempenho da consulta agregada
 18. Roteiro prático de laboratório
 19. Referências
-

1. Objetivos de aprendizagem

Ao final deste módulo, o participante deverá ser capaz de:

- Empregar corretamente SUM, COUNT, AVG, MIN, MAX e as demais funções de agregação do T-SQL, conhecendo o tratamento que cada uma dá a NULL;
 - Escrever consultas com GROUP BY sobre resultados de junção, respeitando a regra de compatibilidade entre colunas agregadas e não agregadas;
 - Distinguir com segurança o papel do ON, do WHERE e do HAVING numa mesma consulta;
 - Reconhecer e neutralizar o *fan-out* — a duplicação de linhas que inflaciona somatórios em junções 1:N;
 - Preservar grupos vazios em junções externas, devolvendo zero onde o correto é zero e NULL onde o correto é "sem informação";
 - Produzir subtotais hierárquicos com ROLLUP e GROUPING SETS para relatórios por região fiscal e município;
 - Construir os indicadores de risco usuais da malha fiscal — carga tributária efetiva, índice de cancelamento, volumetria, ticket médio e concentração — a partir da base `curso_integridade_fiscal`.
-

2. Do documento ao indicador: por que agregar

A base de trabalho tem 10.636 notas fiscais e 30.794 itens. Nenhum auditor lê 30.794 linhas. O que a fiscalização precisa é de **indicadores**: quanto cada contribuinte emitiu, qual a carga efetiva de ICMS que ele declarou, quantas notas ele cancelou, qual a média do valor unitário que ele pratica em cada NCM.

Agregar é reduzir muitas linhas a uma. A cláusula **GROUP BY** define **o critério da redução** — uma linha por contribuinte, por município, por período, por CFOP — e as funções de agregação definem **o que fazer com as linhas de cada grupo**.

A junção entra porque quase nenhum indicador fiscal se calcula com uma tabela só:

Indicador	Tabelas envolvidas	Agregação
Faturamento declarado por município	nfe + contribuinte + municipio	SUM(valor_total)
Carga efetiva de ICMS por contribuinte	nfe + contribuinte	SUM(icms) / SUM(total)
Itens por documento	nfe + nfe_item	COUNT(*) por id_nfe
Divergência NF-e × EFD por período	nfe + efd_c100	SUM dos dois lados
Produtividade do auditor	auditor_fiscal + ordem_servico + auto_infracao	COUNT + SUM(multa)

A frase que resume o módulo: a junção recompõe a informação; a agregação a resume. Errar a junção falsifica o resumo — e um relatório fiscal falso é pior que nenhum relatório.

3. A ordem lógica de processamento da consulta

Antes de qualquer função, é preciso fixar **a ordem em que o SQL Server processa logicamente as cláusulas**. Ela não é a ordem em que escrevemos, e é a partir dela que se explicam todas as regras deste módulo.

Ordem	Cláusula	O que faz
1	FROM / JOIN ... ON	Monta o conjunto combinado; o ON decide quem casa com quem
2	WHERE	Filtra linhas individuais , antes de qualquer agrupamento
3	GROUP BY	Reduz as linhas restantes a grupos
4	HAVING	Filtra grupos , já com o resultado das funções de agregação
5	SELECT	Calcula as expressões e aplica os apelidos (<i>aliases</i>)
6	DISTINCT	Elimina linhas repetidas do resultado
7	ORDER BY	Ordena o resultado final
8	TOP / OFFSET-FETCH	Corta as primeiras linhas

Três consequências práticas desta ordem, todas cobradas em avaliação:

1. O **WHERE** não pode conter função de agregação. No passo 2 os grupos ainda não existem. **WHERE SUM(valor_total) > 1000** é erro de sintaxe.
2. O **HAVING** não pode referenciar colunas fora do **GROUP BY**, exceto dentro de uma função de agregação — no passo 4, cada grupo já perdeu suas linhas individuais.
3. O apelido criado no **SELECT** não existe no **WHERE**, no **GROUP BY** nem no **HAVING**, porque o **SELECT** é processado depois deles. Só o **ORDER BY** (passo 7) enxerga apelidos.

```
-- ERRO: 'carga_efetiva' ainda não existe quando o HAVING é avaliado
SELECT n.id_emitente, SUM(n.valor_icms)/SUM(n.valor_total) AS carga_efetiva
FROM   dbo.nfe AS n
GROUP BY n.id_emitente
HAVING carga_efetiva < 0.04;           -- <<< não compila

-- CORRETO: repita a expressão no HAVING
SELECT n.id_emitente, SUM(n.valor_icms)/SUM(n.valor_total) AS carga_efetiva
FROM   dbo.nfe AS n
GROUP BY n.id_emitente
HAVING SUM(n.valor_icms)/SUM(n.valor_total) < 0.04
ORDER BY carga_efetiva;               -- <<< aqui o apelido funciona
```

4. As funções de agregação

4.1 COUNT — contagem

Três formas, três significados distintos. Confundi-los é o erro mais comum de todo o módulo.

```
SELECT  COUNT(*)                AS linhas_do_grupo,
        COUNT(c.faturamento_declarado) AS com_faturamento,
        COUNT(DISTINCT c.regime_tributario) AS regimes_distintos
FROM    dbo.contribuinte AS c;
-- 60 | 54 | 4
```

Forma	Conta	Ignora NULL?
COUNT(*)	linhas do grupo	Não — conta a linha inteira
COUNT(coluna)	valores não nulos da coluna	Sim
COUNT(DISTINCT coluna)	valores distintos e não nulos	Sim

A diferença entre 60 e 54 são exatamente os **6 contribuintes sem faturamento declarado** da base. Em junção externa, essa diferença deixa de ser curiosidade estatística e passa a ser a distinção entre "zero fiscalizações" e "uma fiscalização" — ver seção 9.

COUNT devolve INT. Para conjuntos acima de 2.147.483.647 linhas existe COUNT_BIG, que devolve BIGINT — irrelevante nesta base, obrigatório em ambientes de produção da SEFAZ.

4.2 SUM — soma

```
-- Valor emitido e ICMS destacado por regime tributário
SELECT  c.regime_tributario,
        COUNT(*)                AS qtd_notas,
        SUM(n.valor_total)      AS valor_emitido,
        SUM(n.valor_icms)       AS icms_destacado
FROM    dbo.nfe AS n
INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE   n.situacao = 'AUTORIZADA'
AND     n.tipo_operacao = '1'
GROUP BY c.regime_tributario
ORDER BY valor_emitido DESC;
```

SUM aceita apenas tipos numéricos, ignora NULL e — atenção — **devolve NULL, não zero, quando o grupo não tem nenhum valor**. Em relatório fiscal, use COALESCE(SUM(...), 0) sempre que o zero for a leitura correta.

O SUM condicional é uma técnica de uso constante na fiscalização, porque permite calcular vários indicadores numa única varredura:

```
-- Composição da emissão por contribuinte, em uma passagem só
SELECT  n.id_emitente,
        COUNT(*) AS qtd_total,
```

```

SUM(CASE WHEN n.situacao = 'AUTORIZADA' THEN 1 ELSE 0 END) AS qtd_autorizadas,
SUM(CASE WHEN n.situacao = 'CANCELADA' THEN 1 ELSE 0 END) AS qtd_canceladas,
SUM(CASE WHEN n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
      THEN n.valor_total ELSE 0 END) AS valor_saidas,
SUM(CASE WHEN n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '0'
      THEN n.valor_total ELSE 0 END) AS valor_entradas
FROM   dbo.nfe AS n
GROUP BY n.id_emitente;

```

4.3 AVG — média

```

-- Ticket médio por contribuinte emitente
SELECT  c.cnpj, c.razao_social,
        COUNT(*) AS qtd_notas,
        AVG(n.valor_total) AS ticket_medio,
        SUM(n.valor_total) AS total_emitido
FROM     dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE    n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
GROUP BY c.cnpj, c.razao_social
HAVING   COUNT(*) >= 30
ORDER BY ticket_medio DESC;

```

Duas armadilhas de **AVG**:

(a) **AVG ignora NULL — e isso muda o denominador.**

```

SELECT  AVG(c.faturamento_declarado) AS media_ignorando_nulos, -- /54
        SUM(c.faturamento_declarado) / COUNT(*) AS media_sobre_60, -- /60
        AVG(COALESCE(c.faturamento_declarado, 0)) AS media_tratando_nulo -- /60
FROM     dbo.contribuinte AS c;

```

As três estão corretas *matematicamente*; apenas uma responde à pergunta que você fez. Se a pergunta é "qual o faturamento médio dos contribuintes que declararam", use a primeira. Se é "qual o faturamento médio do cadastro", o não declarante entra no denominador — e aí a segunda ou a terceira.

(b) **AVG sobre inteiro faz divisão inteira.**

```

SELECT  AVG(quantidade_inteira) ... -- trunca!
SELECT  AVG(CAST(coluna AS DECIMAL(15,4))) ... -- correto

```

Na base do curso os valores monetários são **DECIMAL**, então o problema não aparece — mas aparece o tempo todo ao agregar colunas de contagem.

4.4 MIN e MAX — extremos

Funcionam com números, datas e texto, e ignoram **NULL**.

```

-- Janela temporal e faixa de valores de cada contribuinte
SELECT  c.cnpj, c.razao_social,
        MIN(n.data_emissao) AS primeira_emissao,
        MAX(n.data_emissao) AS ultima_emissao,
        DATEDIFF(DAY, MIN(n.data_emissao), MAX(n.data_emissao)) AS dias_de_atividade,
        MIN(n.valor_total) AS menor_nota,
        MAX(n.valor_total) AS maior_nota
FROM     dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE    n.situacao = 'AUTORIZADA'
GROUP BY c.cnpj, c.razao_social
ORDER BY ultima_emissao DESC;

```

⚠ **MAX não traz a linha do máximo, apenas o valor.** Para saber *qual* nota tem o maior valor, **MAX** não serve: use **CROSS APPLY ... TOP (1)** (visto no módulo de junções) ou **ROW_NUMBER()**. Escrever **SELECT MAX(valor_total), chave_acesso ... GROUP BY chave_acesso** devolve uma linha por chave — que é o oposto do pretendido.

4.5 STDEV, STDEVP, VAR, VARP — dispersão

Desvio padrão e variância, amostral (**STDEV**, **VAR**) e populacional (**STDEVP**, **VARP**). Na malha fiscal servem para localizar **comportamento atípico**: o contribuinte cujo preço unitário oscila muito mais que a média do setor merece atenção.

```
-- Dispersão do preço unitário praticado por NCM
SELECT i.ncm,
       COUNT(*) AS qtd_itens,
       AVG(i.valor_unitario) AS preco_medio,
       STDEV(i.valor_unitario) AS desvio_padrao,
       STDEV(i.valor_unitario) / NULLIF(AVG(i.valor_unitario),0) AS coef_variacao
FROM   dbo.nfe_item AS i
       INNER JOIN dbo.nfe AS n ON n.id_nfe = i.id_nfe
WHERE  n.situacao = 'AUTORIZADA'
GROUP BY i.ncm
HAVING COUNT(*) >= 50
ORDER BY coef_variacao DESC;
```

STDEV devolve **NULL** quando o grupo tem uma única linha — não há dispersão amostral de um ponto só. O **HAVING COUNT(*) >= 50** acima protege contra grupos pequenos, que produzem estatística sem significado.

4.6 STRING_AGG — concatenação (SQL Server 2017+)

Agrega texto em vez de número. Útil para consolidar, numa linha só, a lista de dispositivos legais infringidos ou de auditores envolvidos.

```
SELECT c.cnpj, c.razao_social,
       COUNT(*) AS qtd_autos,
       SUM(a.valor_principal + a.valor_multa) AS credito_lancado,
       STRING_AGG(a.numero_auto, ', ') WITHIN GROUP (ORDER BY a.data_lavratura)
       AS autos_lavrados
FROM   dbo.auto_infracao AS a
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = a.id_contribuinte
GROUP BY c.cnpj, c.razao_social
ORDER BY credito_lancado DESC;
```

4.7 As demais

- **APPROX_COUNT_DISTINCT** (SQL Server 2019+): contagem aproximada de distintos, com erro médio em torno de 2%, muito mais barata em memória. Útil sobre bilhões de chaves de acesso; desnecessária nesta base.
- **CHECKSUM_AGG**: devolve o *checksum* de um grupo de valores inteiros. Serve para detectar rapidamente se um conjunto mudou entre duas cargas — não é função criptográfica e admite colisão.
- **GROUPING** e **GROUPING_ID**: não agregam dados; identificam se a linha atual é uma linha de subtotal gerada por **ROLLUP** ou **CUBE** (seção 13).
- **COUNT_BIG**: idêntica a **COUNT**, com retorno **BIGINT**.

5. Quadro resumo de todas as funções de agregação

Função	Devolve	Tipos aceitos	Ignora NULL?	Aceita DISTINCT?	Grupo vazio devolve	Uso típico na fiscalização
COUNT(*)	INT	—	Não	Não	0	Volume de documentos do grupo

Função	Devolve	Tipos aceitos	Ignora NULL?	Aceita DISTINCT?	Grupo vazio devolve	Uso típico na fiscalização
COUNT(coluna)	INT	qualquer	Sim	Sim	0	Quantos têm o dado preenchido
COUNT(DISTINCT col)	INT	qualquer	Sim	—	0	Nº de contribuintes/NCM distintos
COUNT_BIG(...)	BIGINT	qualquer	igual a COUNT	Sim	0	Contagem acima de 2,1 bilhões
APPROX_COUNT_DISTINCT	BIGINT	qualquer	Sim	—	0	Distintos aproximados em base massiva (2019+)
SUM	tipo da coluna	numérico	Sim	Sim	NULL	Valor emitido, ICMS, multa
AVG	tipo da coluna	numérico	Sim	Sim	NULL	Ticket médio, alíquota média
MIN	tipo da coluna	numérico, data, texto	Sim	irrelevante	NULL	Primeira emissão, menor preço
MAX	tipo da coluna	numérico, data, texto	Sim	irrelevante	NULL	Última emissão, maior nota
STDEV	FLOAT	numérico	Sim	Sim	NULL	Dispersão de preço (amostral)
STDEVP	FLOAT	numérico	Sim	Sim	NULL	Dispersão (populacional)
VAR	FLOAT	numérico	Sim	Sim	NULL	Variância amostral
VARP	FLOAT	numérico	Sim	Sim	NULL	Variância populacional
STRING_AGG	texto	texto	Sim	Não	NULL	Lista de autos, CFOP, dispositivos (2017+)
CHECKSUM_AGG	INT	inteiro	Sim	Sim	NULL	Detectar alteração entre cargas
GROUPING	TINYINT	coluna do GROUP BY	—	Não	—	Marcar linhas de subtotal
GROUPING_ID	INT	lista de colunas	—	Não	—	Identificar o nível do subtotal

Regras que valem para todas elas:

1. Todas ignoram NULL, **exceto COUNT(*)**.
2. Nenhuma pode aparecer no WHERE — apenas no SELECT, no HAVING e no ORDER BY.
3. Não podem ser aninhadas diretamente: SUM(COUNT(*)) só é válido com função de janela ou subconsulta.
4. SUM, AVG, MIN, MAX, STDEV e STRING_AGG devolvem NULL — e não zero — para grupo sem valores. Em junção externa isso é frequente; trate com COALESCE.

6. Agregação e NULL

6.1 A demonstração numérica

```
SELECT COUNT(*) AS total_contribuintes, -- 60
COUNT(c.faturamento_declarado) AS declararam, -- 54
COUNT(*) - COUNT(c.faturamento_declarado) AS nao_declararam, -- 6
AVG(c.faturamento_declarado) AS media_dos_declarantes,
AVG(COALESCE(c.faturamento_declarado, 0)) AS media_do_cadastro
FROM dbo.contribuinte AS c;
```

6.2 Onde isso vira problema fiscal

Duas colunas da base são anuláveis por decisão de projeto, e ambas afetam agregações:

Coluna	NULL significa	Efeito na agregação
nfe.id_destinatario	consumidor não identificado (1.041 notas)	Agrupar por destinatário cria um grupo NULL — que é legítimo e precisa de rótulo
contribuinte.faturamento_declarado	não declarado (6 contribuintes)	Some do denominador de AVG e do numerador de SUM
municipio.regiao_fiscal	município de outra UF (10 casos)	Grupo NULL no relatório por região
ordem_servico.data_conclusao	fiscalização em andamento (63 casos)	MAX(data_conclusao) ignora as abertas

```
-- Agrupando por destinatário: o grupo NULL precisa de rótulo explícito
SELECT COALESCE(CAST(n.id_destinatario AS VARCHAR(10)), 'CONSUMIDOR NAO IDENTIFICADO')
AS destinatario,
COUNT(*) AS qtd_notas,
SUM(n.valor_total) AS valor
FROM dbo.nfe AS n
WHERE n.situacao = 'AUTORIZADA'
GROUP BY n.id_destinatario
ORDER BY valor DESC;
```

Importante: ao contrário do que acontece nas junções, o **GROUP BY** agrupa os **NULL** juntos. Todos os **NULL** formam um único grupo. É a única cláusula do SQL em que **NULL** é tratado como igual a **NULL**.

7. GROUP BY: a regra de ouro

Toda coluna que aparece no **SELECT** fora de uma função de agregação precisa constar do **GROUP BY**.

```
-- ERRO: 'razao_social' não está agregada nem agrupada
SELECT c.id_contribuinte, c.razao_social, COUNT(*)
FROM dbo.contribuinte AS c
INNER JOIN dbo.nfe AS n ON n.id_emitente = c.id_contribuinte
GROUP BY c.id_contribuinte;

-- CORRETO
GROUP BY c.id_contribuinte, c.razao_social;
```

Alguns SGBDs relaxam essa regra quando a coluna é funcionalmente dependente da chave agrupada; o **SQL Server** não relaxa. Inclua a coluna no **GROUP BY** — o custo é desprezível, já que **id_contribuinte** é

chave primária e não cria grupos adicionais.

7.1 Agrupamento por múltiplas colunas

Cada combinação distinta de valores forma um grupo:

```
-- Uma linha por (região fiscal, regime tributário, situação da nota)
SELECT  m.regiao_fiscal,
        c.regime_tributario,
        n.situacao,
        COUNT(*)           AS qtd,
        SUM(n.valor_total) AS valor
FROM    dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
        INNER JOIN dbo.municipio   AS m ON m.id_municipio   = c.id_municipio
GROUP BY m.regiao_fiscal, c.regime_tributario, n.situacao
ORDER BY m.regiao_fiscal, c.regime_tributario, n.situacao;
```

7.2 Agrupamento por expressão

O **GROUP BY** aceita expressões, desde que a **mesma expressão** apareça no **SELECT**:

```
-- Emissão por competência (ano-mês), a partir da data
SELECT  FORMAT(n.data_emissao, 'yyyy-MM') AS competencia,
        COUNT(*)                         AS qtd_notas,
        SUM(n.valor_total)               AS valor_emitido,
        SUM(n.valor_icms)                AS icms
FROM    dbo.nfe AS n
WHERE   n.situacao = 'AUTORIZADA'
GROUP BY FORMAT(n.data_emissao, 'yyyy-MM')
ORDER BY competencia;
```

Alternativas mais eficientes que **FORMAT** (que é notoriamente lenta em volume alto):

```
GROUP BY YEAR(n.data_emissao), MONTH(n.data_emissao)
GROUP BY DATEFROMPARTS(YEAR(n.data_emissao), MONTH(n.data_emissao), 1)
GROUP BY CONVERT(CHAR(6), n.data_emissao, 112) -- AAAAMM, casa com efd_c100.periodo_apuracao
```

A última é particularmente útil nesta base: produz o mesmo formato **AAAAMM** da coluna **periodo_apuracao** da EFD, permitindo cruzar diretamente os dois lados.

7.3 Agrupamento por **CASE**: faixas de valor

```
-- Distribuição das notas por faixa de valor (perfil de emissão)
SELECT  CASE WHEN n.valor_total < 100 THEN '1. ate 100'
            WHEN n.valor_total < 1000 THEN '2. 100 a 1.000'
            WHEN n.valor_total < 10000 THEN '3. 1.000 a 10.000'
            WHEN n.valor_total < 100000 THEN '4. 10.000 a 100.000'
            ELSE '5. acima de 100.000'
        END AS faixa,
        COUNT(*)           AS qtd_notas,
        SUM(n.valor_total) AS valor_total,
        AVG(n.valor_total) AS ticket_medio
FROM    dbo.nfe AS n
WHERE   n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
GROUP BY CASE WHEN n.valor_total < 100 THEN '1. ate 100'
            WHEN n.valor_total < 1000 THEN '2. 100 a 1.000'
            WHEN n.valor_total < 10000 THEN '3. 1.000 a 10.000'
            WHEN n.valor_total < 100000 THEN '4. 10.000 a 100.000'
            ELSE '5. acima de 100.000'
        END
ORDER BY faixa;
```

Repetir o **CASE** inteiro é obrigatório (o apelido não existe no **GROUP BY**). Para evitar a duplicação, use uma CTE ou subconsulta que crie a coluna antes:


```

WITH classificado AS (
    SELECT n.valor_total,
           CASE WHEN n.valor_total < 100 THEN '1. ate 100' ELSE '2. acima' END AS faixa
    FROM   dbo.nfe AS n
    WHERE  n.situacao = 'AUTORIZADA'
)
SELECT faixa, COUNT(*) AS qtd, SUM(valor_total) AS valor
FROM   classificado
GROUP BY faixa;

```

8. Agregação sobre junção interna

A junção acontece **antes** do agrupamento (passo 1 contra passo 3). Portanto, agregar sobre junção interna significa agregar sobre o conjunto **já filtrado pela correspondência** — as linhas sem par nunca chegam ao `GROUP BY`.

8.1 Exemplo completo: painel por município

```

SELECT m.regiao_fiscal,
       m.nome AS municipio,
       COUNT(DISTINCT c.id_contribuinte) AS qtd_contribuintes,
       COUNT(*) AS qtd_notas,
       SUM(n.valor_total) AS valor_emitido,
       SUM(n.valor_icms) AS icms_destacado,
       AVG(n.valor_total) AS ticket_medio,
       MAX(n.valor_total) AS maior_nota
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
       INNER JOIN dbo.municipio AS m ON m.id_municipio = c.id_municipio
WHERE  n.situacao = 'AUTORIZADA'
       AND n.tipo_operacao = '1'
       AND m.uf = 'PB'
GROUP BY m.regiao_fiscal, m.nome
ORDER BY valor_emitido DESC;

```

Note o `COUNT(DISTINCT c.id_contribuinte)`: como o grupo é o município e cada contribuinte tem várias notas, `COUNT(c.id_contribuinte)` contaria notas, não contribuintes.

8.2 Agregação sobre três níveis: item, documento e cadastro

```

-- Participação de cada CFOP no valor emitido, por regime tributário
SELECT c.regime_tributario,
       i.cfop,
       COUNT(DISTINCT n.id_nfe) AS qtd_documentos,
       COUNT(*) AS qtd_itens,
       SUM(i.valor_total_item) AS valor_itens,
       SUM(i.valor_icms_item) AS icms_itens,
       AVG(i.aliquota_icms) AS aliquota_media
FROM   dbo.nfe_item AS i
       INNER JOIN dbo.nfe AS n ON n.id_nfe = i.id_nfe
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE  n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
GROUP BY c.regime_tributario, i.cfop
ORDER BY c.regime_tributario, valor_itens DESC;

```

Aqui a agregação é **do item**, e por isso `SUM(i.valor_total_item)` está correto. O que **não** se pode fazer nesta mesma consulta é somar `n.valor_total` — ver seção 10.

`AVG(i.aliquota_icms)` merece um alerta: a coluna é anulável (`NULL` quando o item não é tributado), então a média é calculada apenas sobre os itens tributados. Se a intenção é a alíquota média da operação inteira, o correto é `SUM(i.valor_icms_item) / NULLIF(SUM(i.valor_total_item), 0)` — uma média **ponderada**, não aritmética.

9. Agregação sobre junção externa

Este é o ponto em que a agregação e as junções se encontram de forma mais delicada.

9.1 COUNT(*) mente na junção externa

```
-- ERRADO: contribuintes sem fiscalização aparecem com 1
SELECT  c.cnpj, c.razao_social, COUNT(*) AS qtd_os
FROM    dbo.contribuinte AS c
        LEFT JOIN dbo.ordem_servico AS o ON o.id_contribuinte = c.id_contribuinte
GROUP BY c.cnpj, c.razao_social;

-- CORRETO: quem não tem ordem de serviço aparece com 0
SELECT  c.cnpj, c.razao_social, COUNT(o.id_os) AS qtd_os
FROM    dbo.contribuinte AS c
        LEFT JOIN dbo.ordem_servico AS o ON o.id_contribuinte = c.id_contribuinte
GROUP BY c.cnpj, c.razao_social
ORDER BY qtd_os;
```

O **LEFT JOIN** preserva a linha do contribuinte sem par, preenchendo **o.*** com **NULL**. **COUNT(*)** conta essa linha fantasma e devolve 1; **COUNT(o.id_os)** ignora o **NULL** e devolve 0. **Em relatório de fiscalização, a diferença entre 0 e 1 é a diferença entre "nunca fiscalizado" e "já fiscalizado"**.

Regra prática: **em junção externa, conte sempre uma coluna NOT NULL da tabela preservada do lado direito** — normalmente a chave primária.

9.2 SUM devolve NULL, não zero

```
SELECT  c.cnpj, c.razao_social,
        COUNT(a.id_auto) AS qtd_autos,
        SUM(a.valor_principal + a.valor_multa) AS credito_lancado, -- NULL
        COALESCE(SUM(a.valor_principal + a.valor_multa), 0) AS credito_zero -- 0
FROM    dbo.contribuinte AS c
        LEFT JOIN dbo.auto_infracao AS a ON a.id_contribuinte = c.id_contribuinte
GROUP BY c.cnpj, c.razao_social
ORDER BY credito_zero DESC;
```

Qual das duas colunas usar depende da semântica: **NULL** diz "não há informação"; **0** diz "há informação e ela é zero". Para contribuinte nunca autuado, **zero é a leitura correta** — ele deve constar do relatório com crédito lançado de R\$ 0,00, e não com uma célula vazia que o leitor interpretará como falha do sistema.

9.3 Painel completo de produtividade do auditor

```
SELECT  au.matricula,
        au.nome AS auditor,
        au.cargo,
        au.regiao_fiscal,
        sup.nome AS supervisor,
        COUNT(DISTINCT o.id_os) AS os_designadas,
        COUNT(DISTINCT CASE WHEN o.situacao = 'CONCLUIDA'
                              THEN o.id_os END) AS os_concluidas,
        COUNT(DISTINCT a.id_auto) AS autos_lavrados,
        COALESCE(SUM(a.valor_principal), 0) AS principal_lancado,
        COALESCE(SUM(a.valor_multa), 0) AS multa_lancada,
        MIN(o.data_abertura) AS primeira_os,
        MAX(o.data_abertura) AS ultima_os
FROM    dbo.auditor_fiscal AS au
        LEFT JOIN dbo.auditor_fiscal AS sup ON sup.matricula = au.matricula_supervisor
        LEFT JOIN dbo.ordem_servico AS o ON o.matricula_auditor = au.matricula
        LEFT JOIN dbo.auto_infracao AS a ON a.id_os = o.id_os
GROUP BY au.matricula, au.nome, au.cargo, au.regiao_fiscal, sup.nome
ORDER BY autos_lavrados DESC, os_designadas DESC;
```

Esta consulta reúne quase tudo do módulo: autojunção externa para o supervisor, dupla junção externa para preservar os 20 auditores, **COUNT(DISTINCT)** para neutralizar o fan-out entre ordens e autos, **COUNT**

com **CASE** para contagem condicional, e **COALESCE** para transformar ausência em zero.

⚠ **Por que `COUNT(DISTINCT o.id_os)` e não `COUNT(o.id_os)`?** Porque a terceira junção (com `auto_infracao`) repete a linha da ordem de serviço uma vez por auto vinculado. Sem **`DISTINCT`**, uma ordem que gerou três autos seria contada três vezes. É o fan-out da próxima seção, aparecendo dentro de uma agregação.

10. O fan-out: quando a junção falsifica o somatório

10.1 O diagnóstico

```
-- Quantas linhas a junção produz, e quantos documentos existem de fato?
SELECT  COUNT(*)                AS linhas_apos_juncao,
        COUNT(DISTINCT n.id_nfe) AS documentos_distintos
FROM    dbo.nfe AS n
        INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe;
```

Como há 10.636 notas e 30.794 itens, a junção produz 30.794 linhas — quase três por documento. **Toda coluna de `nfe` somada depois dessa junção sai multiplicada.**

```
-- ERRADO: valor_total repetido uma vez por item
SELECT SUM(n.valor_total) FROM dbo.nfe AS n
        INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe;

-- CORRETO: sem a junção, ou com a agregação feita antes
SELECT SUM(n.valor_total) FROM dbo.nfe AS n;
```

10.2 O padrão correto: agregar antes de juntar

Quando o relatório precisa de métricas de **granularidades diferentes** — documentos de um lado, itens do outro — a solução é consolidar cada uma na sua granularidade e só então juntar:

```
WITH doc AS (
    -- granularidade: um documento
    SELECT  n.id_emitente,
            COUNT(*)                AS qtd_documentos,
            SUM(n.valor_total)      AS valor_documentos,
            SUM(n.valor_icms)      AS icms_documentos
    FROM    dbo.nfe AS n
    WHERE   n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
    GROUP BY n.id_emitente
),
item AS (
    -- granularidade: um item
    SELECT  n.id_emitente,
            COUNT(*)                AS qtd_itens,
            SUM(i.valor_total_item) AS valor_itens,
            COUNT(DISTINCT i.ncm)   AS ncm_distintos
    FROM    dbo.nfe AS n
            INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
    WHERE   n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
    GROUP BY n.id_emitente
)
SELECT  c.cnpj, c.razao_social,
        COALESCE(d.qtd_documentos, 0) AS qtd_documentos,
        COALESCE(d.valor_documentos, 0) AS valor_documentos,
        COALESCE(it.qtd_itens, 0) AS qtd_itens,
        COALESCE(it.valor_itens, 0) AS valor_itens,
        it.ncm_distintos,
        CAST(it.qtd_itens AS DECIMAL(10,2)) / NULLIF(d.qtd_documentos,0) AS itens_por_documento,
        d.valor_documentos - it.valor_itens AS diferenca_cabecalho_itens
FROM    dbo.contribuinte AS c
        LEFT JOIN doc AS d ON d.id_emitente = c.id_contribuinte
        LEFT JOIN item AS it ON it.id_emitente = c.id_contribuinte
ORDER BY ABS(COALESCE(d.valor_documentos,0) - COALESCE(it.valor_itens,0)) DESC;
```

A última coluna é, ela própria, uma regra de auditoria: a soma dos itens tem de bater com a soma dos cabeçalhos. A base tem **15 notas** em que isso não ocorre.

10.3 A alternativa: subconsulta escalar correlacionada

Para uma ou duas métricas, a subconsulta no **SELECT** é mais curta e imune a fan-out:

```
SELECT c.cnpj, c.razao_social,
       (SELECT COUNT(*) FROM dbo.nfe AS n
        WHERE n.id_emitente = c.id_contribuinte AND n.situacao = 'AUTORIZADA')
       AS qtd_notas,
       (SELECT COALESCE(SUM(n.valor_total),0) FROM dbo.nfe AS n
        WHERE n.id_emitente = c.id_contribuinte AND n.situacao = 'AUTORIZADA')
       AS valor_emitido
FROM   dbo.contribuinte AS c
ORDER BY valor_emitido DESC;
```

Legível e correto, mas cada subconsulta é uma varredura adicional. Para muitas métricas, prefira o padrão de CTEs da seção 10.2 — uma passagem por tabela, e não uma por coluna.

11. HAVING: o filtro do grupo

11.1 O conceito

HAVING é para grupos o que **WHERE** é para linhas. Ele é avaliado **depois** do **GROUP BY**, quando cada grupo já foi reduzido a uma linha e as funções de agregação já produziram seus valores.

```
-- Emitentes com mais de 1.000 notas autorizadas: sinal de alta volumetria
SELECT n.id_emitente,
       c.cnpj,
       c.razao_social,
       COUNT(*) AS qtd_notas,
       SUM(n.valor_total) AS valor_emitido
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE  n.situacao = 'AUTORIZADA'
GROUP BY n.id_emitente, c.cnpj, c.razao_social
HAVING COUNT(*) > 1000
ORDER BY qtd_notas DESC;
```

Resultado esperado nesta base: **3 emitentes** (**id_contribuinte** 1, 2 e 3). Baixando o limiar para 500, são **6**.

11.2 HAVING com várias condições

As condições se combinam com **AND** / **OR**, como no **WHERE**:

```
-- Índice de cancelamento acima de 15%, entre quem tem volume relevante
SELECT n.id_emitente,
       c.razao_social,
       COUNT(*) AS qtd_notas,
       SUM(CASE WHEN n.situacao = 'CANCELADA' THEN 1 ELSE 0 END) AS canceladas,
       CAST(SUM(CASE WHEN n.situacao='CANCELADA' THEN 1 ELSE 0 END) AS DECIMAL(10,2))
       / NULLIF(COUNT(*),0) * 100 AS pct_cancelamento
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
GROUP BY n.id_emitente, c.razao_social
HAVING COUNT(*) >= 100
       AND CAST(SUM(CASE WHEN n.situacao='CANCELADA' THEN 1 ELSE 0 END) AS DECIMAL(10,2))
       / NULLIF(COUNT(*),0) * 100 > 15
ORDER BY pct_cancelamento DESC;
```

Resultado esperado: **2 contribuintes** (**id** 4 e 12).

O **HAVING COUNT(*) >= 100** cumpre papel metodológico, não decorativo: sem ele, um contribuinte com 4 notas e 1 cancelada apareceria com 25% de cancelamento e seria selecionado para fiscalização sem nenhuma base estatística. **Tudo indicador percentual precisa de um piso de volume no HAVING.**

11.3 HAVING sem GROUP BY

É válido: a consulta inteira vira um único grupo.

```
-- Devolve a linha apenas se o total emitido superar 1 bilhão
SELECT SUM(n.valor_total) AS total_geral
FROM   dbo.nfe AS n
WHERE  n.situacao = 'AUTORIZADA'
HAVING SUM(n.valor_total) > 1000000000;
```

Raro na prática, mas útil em rotinas de alerta.

11.4 O que não se pode fazer no HAVING

```
-- ERRO 1: apelido do SELECT não existe no HAVING
HAVING qtd_notas > 1000

-- ERRO 2: coluna que não está no GROUP BY e não está agregada
SELECT n.id_emitente, COUNT(*) FROM dbo.nfe AS n
GROUP BY n.id_emitente
HAVING n.valor_total > 1000;           -- 'valor_total' não existe mais no nível do grupo

-- CORRETO, se a intenção era filtrar linhas antes de agrupar:
WHERE  n.valor_total > 1000

-- CORRETO, se a intenção era filtrar grupos:
HAVING MAX(n.valor_total) > 1000
```

12. WHERE × HAVING × ON: os três filtros

12.1 A distinção conceitual

Cláusula	Filtra	Quando age	Aceita agregação?	Pergunta que responde
ON	pares de linhas	durante a junção	Não	"Quem casa com quem?"
WHERE	linhas individuais	antes do agrupamento	Não	"Quais documentos entram na conta?"
HAVING	grupos	depois do agrupamento	Sim	"Quais contribuintes o resultado deve reportar?"

12.2 O exemplo que diferencia WHERE de HAVING

O par de consultas abaixo tem a **mesma estrutura** e o **mesmo número**, 1000. Os resultados são completamente diferentes, e a diferença é apenas a cláusula em que o predicado foi escrito.

```
/* CONSULTA A - o filtro está no WHERE: filtra NOTAS */
SELECT  c.cnpj,
        c.razao_social,
        COUNT(*)           AS qtd_notas,
        SUM(n.valor_total) AS valor_emitido
FROM    dbo.nfe AS n
INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE   n.situacao = 'AUTORIZADA'
```

```

AND n.valor_total > 1000 -- <<< descarta a NOTA de valor baixo
GROUP BY c.cnpj, c.razao_social
ORDER BY valor_emitido DESC;

```

Leitura de A: "Considerando ****apenas as notas acima de R1.000 ***

***, quantocadacontribuinteemituiu?" Asnotaspequenassãodescartadasantesdasoma. Aparecemnoresultadotodo**
1.000 — e os valores somados **excluem o movimento de baixo valor.**

```

/* CONSULTA B - o filtro está no HAVING: filtra CONTRIBUINTES */
SELECT c.cnpj,
       c.razao_social,
       COUNT(*) AS qtd_notas,
       SUM(n.valor_total) AS valor_emitido
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE  n.situacao = 'AUTORIZADA'
GROUP BY c.cnpj, c.razao_social
HAVING SUM(n.valor_total) > 1000 -- <<< descarta o CONTRIBUINTE de soma baixa
ORDER BY valor_emitido DESC;

```

Leitura de B: "Considerando **todas** as notas autorizadas, quais contribuintes acumularam mais de R\$ 1.000 no total?" Nenhuma nota é descartada; o que é descartado é o contribuinte cujo **somatório** não atinge o limiar.

	Consulta A (WHERE)	Consulta B (HAVING)
O que é eliminado	notas individuais	contribuintes inteiros
Momento da eliminação	antes de somar	depois de somar
SUM(valor_total) representa	soma parcial (só notas > 1.000)	soma integral
COUNT(*) representa	notas grandes do contribuinte	todas as notas do contribuinte
Aceita função de agregação	não	sim

12.3 As duas juntas — o caso mais comum na prática

Quase todo indicador fiscal real usa as três cláusulas, cada uma no seu papel:

```

SELECT c.cnpj,
       c.razao_social,
       m.regiao_fiscal,
       COUNT(*) AS qtd_notas,
       SUM(n.valor_total) AS base_declarada,
       SUM(n.valor_icms) AS icms_declarado,
       SUM(n.valor_icms) / NULLIF(SUM(n.valor_total),0) * 100 AS carga_efetiva_pct
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c
           ON c.id_contribuinte = n.id_emitente -- ON: quem casa com quem
       INNER JOIN dbo.municipio AS m
           ON m.id_municipio = c.id_municipio
WHERE  n.situacao = 'AUTORIZADA' -- WHERE: quais notas contam
       AND n.tipo_operacao = '1'
       AND c.regime_tributario = 'NORMAL'
GROUP BY c.cnpj, c.razao_social, m.regiao_fiscal
HAVING SUM(n.valor_total) > 100000 -- HAVING: quais contribuintes reportar
       AND SUM(n.valor_icms) / NULLIF(SUM(n.valor_total),0) < 0.04
ORDER BY carga_efetiva_pct;

```

Este é o indicador de **carga tributária efetiva abaixo do esperado**: contribuintes do regime normal, com movimento relevante, cujo ICMS destacado representa menos de 4% da base. Resultado esperado nesta base: **2 contribuintes** (id 5 e 17).

Repare que **c.regime_tributario = 'NORMAL'** está no **WHERE**, e não no **HAVING**, porque é um atributo da **linha**, não do grupo. Colocá-lo no **HAVING** seria erro de sintaxe (a coluna não está agregada); colocá-lo no **ON** da junção com **contribuinte** funcionaria, mas confundiria o leitor: em junção interna, condição de negócio pertence ao **WHERE**.

13. Subtotais: ROLLUP, CUBE e GROUPING SETS

Relatórios fiscais quase sempre pedem hierarquia: total por município, subtotal por região fiscal, total geral. Escrever três consultas unidas por **UNION ALL** funciona, mas percorre a base três vezes. As extensões do **GROUP BY** fazem isso em uma passagem.

13.1 ROLLUP — subtotais hierárquicos

```
SELECT COALESCE(m.regiao_fiscal, 'TOTAL GERAL') AS regiao_fiscal,
       COALESCE(m.nome, 'SUBTOTAL DA REGIAO') AS municipio,
       COUNT(DISTINCT c.id_contribuinte) AS contribuintes,
       COUNT(*) AS qtd_notas,
       SUM(n.valor_total) AS valor_emitido,
       GROUPING(m.nome) AS eh_subtotal_regiao,
       GROUPING(m.regiao_fiscal) AS eh_total_geral
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
       INNER JOIN dbo.municipio AS m ON m.id_municipio = c.id_municipio
WHERE  n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1' AND m.uf = 'PB'
GROUP BY ROLLUP (m.regiao_fiscal, m.nome)
ORDER BY GROUPING(m.regiao_fiscal), m.regiao_fiscal, GROUPING(m.nome), m.nome;
```

O **ROLLUP (a, b)** produz três níveis: um por **(a, b)**, um por **(a)** — o subtotal — e um único para o total geral. A função **GROUPING(coluna)** devolve **1** quando a coluna foi "colapsada" naquela linha, permitindo distinguir um subtotal de um grupo cujo valor é genuinamente **NULL** — distinção indispensável nesta base, onde **regiao_fiscal** é nula para os 10 municípios de outras UFs.

13.2 CUBE — todas as combinações

CUBE (a, b) produz os subtotais de **(a,b)**, **(a)**, **(b)** e do total geral. Útil quando as dimensões não são hierárquicas:

```
SELECT COALESCE(c.regime_tributario, 'TODOS OS REGIMES') AS regime,
       COALESCE(n.situacao, 'TODAS AS SITUACOES') AS situacao,
       COUNT(*) AS qtd,
       SUM(n.valor_total) AS valor
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
GROUP BY CUBE (c.regime_tributario, n.situacao)
ORDER BY regime, situacao;
```

13.3 GROUPING SETS — os subtotais que você escolher

```
-- Exatamente três blocos: por região, por regime, e total geral
SELECT m.regiao_fiscal,
       c.regime_tributario,
       COUNT(*) AS qtd_notas,
       SUM(n.valor_total) AS valor
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
       INNER JOIN dbo.municipio AS m ON m.id_municipio = c.id_municipio
WHERE  n.situacao = 'AUTORIZADA'
GROUP BY GROUPING SETS (
    (m.regiao_fiscal),
    (c.regime_tributario),
    () -- total geral
);
```

ROLLUP e **CUBE** são casos particulares de **GROUPING SETS**. Quando o relatório precisa de subtotais específicos, e não de toda a hierarquia, **GROUPING SETS** é a forma mais econômica.

14. Agregação × funções de janela

Existe uma diferença conceitual que costuma confundir e vale registrar desde já:

	GROUP BY + agregação	Função de janela (OVER)
Efeito sobre as linhas	reduz o conjunto	preserva todas as linhas
Uma linha por	grupo	linha original
Sintaxe	SUM(x) ... GROUP BY y	SUM(x) OVER (PARTITION BY y)
Uso típico	painel consolidado	percentual de participação linha a linha

```
-- Cada nota, com o total do seu emitente ao lado e a participação percentual
SELECT TOP (50)
    n.chave_acesso,
    n.valor_total,
    SUM(n.valor_total) OVER (PARTITION BY n.id_emitente) AS total_do_emitente,
    n.valor_total * 100.0
    / SUM(n.valor_total) OVER (PARTITION BY n.id_emitente) AS pct_participacao,
    COUNT(*) OVER (PARTITION BY n.id_emitente) AS notas_do_emitente
FROM    dbo.nfe AS n
WHERE   n.situacao = 'AUTORIZADA'
ORDER BY pct_participacao DESC;
```

Este resultado — "uma única nota responde por 80% do faturamento anual do contribuinte" — é um indício de risco que o GROUP BY sozinho não expõe, porque ele elimina a linha individual. As funções de janela têm módulo próprio; aqui basta saber **que elas existem e quando o GROUP BY não é a ferramenta certa**.

15. Indicadores fiscais construídos com agregação

Quadro dos indicadores que a malha fiscal calcula sobre esta base, com a fórmula e o resultado conhecido:

Indicador	Fórmula	Filtro de significância	Esperado
Volumetria de emissão	COUNT(*) por emitente	HAVING COUNT(*) > 1000	3 emitentes (id 1, 2, 3)
Volumetria (limiar menor)	COUNT(*) por emitente	HAVING COUNT(*) > 500	6 emitentes
Índice de cancelamento	canceladas / total * 100	HAVING COUNT(*) >= 100	2 contribuintes (id 4, 12)
Carga tributária efetiva	SUM(icms) / SUM(total)	HAVING SUM(total) > 100000	2 contribuintes (id 5, 17)
Consistência cabeçalho × itens	SUM(valor_total_item) vs. valor_total	tolerância R\$ 0,01	15 notas
Divergência NF-e × EFD	SUM dos dois lados por chave	tolerância R\$ 0,01	147 documentos
Omissão de EFD (período 202501)	antijunção + agregação	ATIVO e regime NORMAL	8 contribuintes
Consumidor não identificado	COUNT(*) com id_destinatario IS NULL	—	1.041 notas
Ordens em aberto	COUNT(*) com data_conclusao IS NULL	—	63 ordens

15.1 Concentração de destinatários

Um indicador que combina tudo: contribuintes cujo faturamento se concentra em poucos destinatários — padrão típico de operação simulada.

```
WITH por_par AS (  
    SELECT  n.id_emitente,  
            n.id_destinatario,  
            SUM(n.valor_total) AS valor_par  
    FROM    dbo.nfe AS n  
    WHERE   n.situacao = 'AUTORIZADA'  
    AND     n.tipo_operacao = '1'  
    AND     n.id_destinatario IS NOT NULL  
    GROUP BY n.id_emitente, n.id_destinatario  
)  
por_emitente AS (  
    SELECT  id_emitente,  
            COUNT(*) AS qtd_destinatarios,  
            SUM(valor_par) AS valor_total,  
            MAX(valor_par) AS maior_destinatario  
    FROM    por_par  
    GROUP BY id_emitente  
)  
SELECT  c.cnpj, c.razao_social,  
        e.qtd_destinatarios,  
        e.valor_total,  
        e.maior_destinatario,  
        e.maior_destinatario / NULLIF(e.valor_total,0) * 100 AS pct_concentracao  
FROM    por_emitente AS e  
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = e.id_emitente  
WHERE   e.valor_total > 100000  
ORDER BY pct_concentracao DESC;
```

Note a **agregação em dois estágios**: primeiro por par (emitente, destinatário), depois por emitente. Agregar sobre o resultado de outra agregação é o padrão para responder perguntas do tipo "qual o maior, entre os totais".

16. Erros comuns — quadro de diagnóstico

Sintoma	Causa	Correção
Column ... is invalid in the select list	Coluna no SELECT fora do GROUP BY e sem agregação	Inclua-a no GROUP BY ou agregue-a
Invalid column name no HAVING	Uso de apelido do SELECT	Repita a expressão inteira
An aggregate may not appear in the WHERE clause	SUM/COUNT no WHERE	Mova para o HAVING
Contribuintes sem fiscalização aparecem com contagem 1	COUNT(*) sobre LEFT JOIN	Use COUNT(coluna_da_direita)
Célula vazia onde deveria haver zero	SUM de grupo sem linhas devolve NULL	COALESCE(SUM(...), 0)
Somatório várias vezes maior que o real	Fan-out em junção 1:N	Agregue antes de juntar (seção 10)
Contagem de contribuintes maior que 60	COUNT sem DISTINCT após junção	COUNT(DISTINCT c.id_contribuinte)
Média "estranhamente alta"	AVG ignorando NULL no denominador	Decida entre AVG(col) e AVG(COALESCE(col,0))
Média de percentuais diferente do percentual do total	Média aritmética onde cabia ponderada	SUM(parte)/SUM(todo) , não AVG(parte/todo)
Divisão por zero	Grupo com denominador zerado	NULLIF(denominador, 0)

Sintoma	Causa	Correção
Percentual absurdo em contribuinte pequeno	Falta de piso de volume	<code>HAVING COUNT(*) >= n</code>
Subtotal confundido com grupo nulo	<code>ROLLUP</code> sobre coluna anulável	Use <code>GROUPING(coluna)</code>

17. Desempenho da consulta agregada

1. **Filtre no `WHERE`, não no `HAVING`, sempre que o predicado for de linha.** Filtrar antes reduz o volume que chega ao agrupamento; filtrar depois obriga o SGBD a agregar linhas que serão descartadas.
2. **Índice de cobertura na chave de agrupamento.** O script `08_indices_opcionais.sql` cria `IX_nfe_emitente_data` com `INCLUDE (situacao, tipo_operacao, valor_total, valor_icms)`: exatamente as colunas de um painel por emitente. Com ele, a agregação se resolve no índice, sem tocar na tabela.
3. **`COUNT(*)` não é mais lento que `COUNT(1)`.** É o mesmo plano de execução; a crença contrária é folclore. Já `COUNT(DISTINCT coluna)` é caro — exige ordenação ou hash adicional. Use apenas quando o `DISTINCT` for necessário de fato.
4. **Leia o operador de agregação no plano.** `Stream Aggregate` exige entrada ordenada (barato se um índice já fornece a ordem); `Hash Match (Aggregate)` não exige ordem, mas consome memória e pode derramar para o `tempdb`.
5. **Evite função sobre a coluna agrupada quando houver alternativa.** `GROUP BY FORMAT(data, 'yyyy-MM')` impede o uso do índice sobre `data_emissao`; `GROUP BY YEAR(...), MONTH(...)` costuma sair melhor, e um filtro por faixa (`data_emissao >= ... AND < ...`) no `WHERE` permite a busca por índice.
6. **Agregue uma vez, use várias.** Se o mesmo consolidado alimenta várias colunas do relatório, calcule-o numa CTE e reutilize, em vez de repetir subconsultas escalares.

18. Roteiro prático de laboratório

Duração estimada: 2 horas **Ambiente:** SSMS ou VS Code + extensão MSSQL **Base:** `curso_integridade_fiscal`, carregada com os scripts 01 a 05

Como este laboratório está organizado

Cada atividade parte de uma **regra de seleção da malha fiscal** — o critério que a administração tributária usa para eleger um contribuinte para fiscalização. A regra vem primeiro; a agregação vem como consequência dela.

Seção	Conteúdo
Regra	O critério de seleção, na linguagem da fiscalização
Fundamento	Por que o indicador aponta risco, e qual o limiar de significância
Agregação exigida	As funções e cláusulas que a regra impõe
Rascunho	Consulta parcialmente escrita, com lacunas numeradas <code>/* (n) ... */</code>
Conferência	O número que a consulta correta deve produzir
Análise	Pergunta a responder em comentário, no próprio script

Entrega. Um arquivo `lab_agregacao_<seu_nome>.sql` com as cinco atividades separadas por comentários, lacunas preenchidas e respostas de análise escritas como comentário abaixo de cada

Preparação (5 min).

```
USE curso_integridade_fiscal;
GO
SELECT 'nfe' AS tabela, COUNT(*) AS obtido, 10636 AS esperado FROM dbo.nfe
UNION ALL SELECT 'nfe_item', COUNT(*), 30794 FROM dbo.nfe_item
UNION ALL SELECT 'efd_c100', COUNT(*), 8059 FROM dbo.efd_c100
UNION ALL SELECT 'contribuinte', COUNT(*), 60 FROM dbo.contribuinte
UNION ALL SELECT 'ordem_servico', COUNT(*), 150 FROM dbo.ordem_servico
UNION ALL SELECT 'auto_infracao', COUNT(*), 72 FROM dbo.auto_infracao;
```

Atividade 1 — Carga tributária efetiva abaixo do esperado

Regra. Contribuinte do regime **NORMAL** cujo ICMS destacado nas saídas autorizadas represente menos de 4% do valor total dessas saídas, considerado apenas quem acumule mais de R\$ 100.000 em base, deve ser selecionado para verificação de carga tributária.

Fundamento. A carga efetiva é a razão entre o imposto destacado e a base de cálculo. Uma carga muito abaixo da alíquota nominal pode ter explicação legítima — operações isentas, substituição tributária, exportação — ou pode indicar destaque a menor. O piso de R\$ 100.000 é o filtro de significância: sem ele, um contribuinte com três notas pequenas produziria um percentual sem valor estatístico.

Agregação exigida. **SUM** de duas colunas com razão entre elas, **COUNT** para volumetria, **GROUP BY** por contribuinte, **HAVING** com **duas** condições — o piso de base e o limiar de carga. Atenção ao denominador zero.

Rascunho.

```
/* ATIVIDADE 1 - Carga tributária efetiva ----- */
SELECT  c.cnpj,
        c.razao_social,
        m.nome AS municipio,
        m.regiao_fiscal,
        COUNT(*) AS qtd_notas,
        SUM(n.valor_total) AS base_declarada,
        SUM(n.valor_icms) AS icms_declarado,
        /* (1) a carga efetiva em percentual, protegida contra divisão por zero */
        AS carga_efetiva_pct
FROM    dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
        INNER JOIN dbo.municipio AS m ON /* (2) */
WHERE   n.situacao = 'AUTORIZADA'
        AND n.tipo_operacao = /* (3) indicador de saída */
        AND c.regime_tributario = /* (4) regime obrigado à apuração normal */
GROUP BY /* (5) todas as colunas não agregadas do SELECT */
HAVING  /* (6) piso de significância: base acumulada acima de R$ 100.000 */
        AND /* (7) o limiar da regra: carga inferior a 4% */
ORDER BY carga_efetiva_pct;
```

Em seguida, escreva a consulta de **volumetria** sobre a mesma base:

```
-- Emitentes por faixa de volume de notas autorizadas
SELECT  n.id_emitente, c.razao_social, COUNT(*) AS qtd_notas
FROM    dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON /* (8) */
WHERE   /* (9) */
GROUP BY n.id_emitente, c.razao_social
HAVING  /* (10) mais de 1.000 notas; depois repita com 500 e anote a diferença */
ORDER BY qtd_notas DESC;
```

Conferência.

Verificação	Esperado
Contribuintes com carga efetiva abaixo de 4%	2 (<code>id_contribuinte</code> 5 e 17)
Emitentes com mais de 1.000 notas autorizadas	3 (<code>id_contribuinte</code> 1, 2 e 3)
Emitentes com mais de 500 notas autorizadas	6

Análise. (a) Por que `c.regime_tributario = 'NORMAL'` está no `WHERE` e não no `HAVING`? O que aconteceria se você tentasse movê-lo? (b) Remova o piso da lacuna (6) e execute de novo: quantos contribuintes passam a aparecer, e por que a maioria deles não deveria ser intimada? (c) A carga efetiva calculada como `SUM(icms)/SUM(total)` é diferente de `AVG(valor_icms/valor_total)`. Calcule as duas para o contribuinte 5 e explique qual é a correta.

Atividade 2 — Índice de cancelamento e a distinção `WHERE` × `HAVING`

Regra. *Contribuinte com pelo menos 100 documentos emitidos e índice de cancelamento superior a 15% deve ser selecionado para diligência, por indício de uso de documento fiscal para simulação de operação posteriormente desfeita.*

Fundamento. Cancelamento é procedimento legítimo e previsto, mas cancelar uma nota a cada seis é padrão anômalo. O limiar de 100 documentos é o piso estatístico: um contribuinte com 4 notas e 1 cancelada tem 25% de cancelamento e nenhuma relevância fiscal.

Agregação exigida. `SUM` condicional com `CASE` para contar apenas as canceladas, `COUNT(*)` para o denominador, `HAVING` com duas condições. Esta atividade também exige o **experimento comparativo** entre `WHERE` e `HAVING`.

Rascunho — parte A (o indicador).

```
/* ATIVIDADE 2A - Índice de cancelamento ----- */
SELECT  n.id_emitente,
        c.cnpj,
        c.razao_social,
        COUNT(*) AS qtd_documentos,
        /* (1) contagem apenas das notas em situação CANCELADA, usando SUM + CASE */
        AS qtd_canceladas,
        /* (2) o percentual de cancelamento, com CAST para DECIMAL e NULLIF no
           denominador - lembre que a divisão entre inteiros trunca */
        AS pct_cancelamento
FROM    dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
GROUP BY n.id_emitente, c.cnpj, c.razao_social
HAVING  /* (3) piso de 100 documentos */
        AND /* (4) limiar de 15% - repita aqui a expressão da lacuna (2) */
ORDER BY pct_cancelamento DESC;
```

Rascunho — parte B (o experimento). Execute as duas consultas abaixo e anote a quantidade de linhas e o total emitido de cada uma.

```
/* ATIVIDADE 2B - filtro de LINHA (WHERE) */
SELECT  c.razao_social, COUNT(*) AS qtd_notas, SUM(n.valor_total) AS valor
FROM    dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE   n.situacao = 'AUTORIZADA'
        AND /* (5) apenas notas de valor superior a R$ 1.000 */
GROUP BY c.razao_social;

/* ATIVIDADE 2C - filtro de GRUPO (HAVING) */
SELECT  c.razao_social, COUNT(*) AS qtd_notas, SUM(n.valor_total) AS valor
FROM    dbo.nfe AS n
        INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE   n.situacao = 'AUTORIZADA'
GROUP BY c.razao_social
HAVING  /* (6) apenas contribuintes cuja soma emitida supera R$ 1.000 */;
```

Conferência.

Verificação	Esperado
Contribuintes com cancelamento acima de 15%	2 (<code>id_contribuinte</code> 4 e 12)
<code>qtd_notas</code> de 2B para um mesmo contribuinte	menor que em 2C
<code>SUM(valor_total)</code> de 2B para um mesmo contribuinte	menor que em 2C

Análise. (a) Escolha um contribuinte presente nas duas consultas e escreva, lado a lado, seus números em 2B e em 2C. Explique cada diferença. (b) Uma das duas responde à pergunta "quanto este contribuinte emitiu no ano?" — qual, e por quê? (c) O que acontece se você tentar escrever `WHERE SUM(n.valor_total) > 1000`? Registre a mensagem de erro e explique-a com base na ordem lógica de processamento.

Atividade 3 — Painel de produtividade da fiscalização

Regra. O relatório gerencial deve apresentar os 20 auditores fiscais, com o número de ordens de serviço designadas, o número de ordens concluídas, os autos lavrados e o crédito tributário lançado. Auditor sem ordem designada deve constar do relatório com zero em todas as métricas, e não ser omitido.

Fundamento. É um relatório de gestão, não de risco — mas expõe o erro técnico mais comum da agregação sobre junção externa. Omitir o auditor sem processo, ou apresentá-lo com "1 ordem" por efeito de `COUNT(*)`, invalida o relatório inteiro.

Agregação exigida. Junções externas encadeadas, `COUNT` de coluna (não de `*`), `COUNT(DISTINCT)` para neutralizar a duplicação entre ordens e autos, `COALESCE` para converter ausência em zero, e autojunção externa para trazer o supervisor.

Rascunho.

```

/* ATIVIDADE 3 - Produtividade por auditor ----- */
SELECT  au.matricula,
        au.nome           AS auditor,
        au.cargo,
        au.regiao_fiscal,
        sup.nome          AS supervisor,
        /* (1) ordens designadas - qual função e qual coluna evitam contar 1
           para o auditor sem nenhuma ordem, e evitam contar a mesma ordem
           várias vezes por causa da junção com auto_infracao? */
        AS os_designadas,
        /* (2) ordens concluídas - contagem condicional com CASE, também distinta */
        AS os_concluidas,
        COUNT(DISTINCT a.id_auto)           AS autos_lavrados,
        /* (3) crédito lançado (principal + multa), devolvendo 0 e não NULL */
        AS credito_lancado,
        MIN(o.data_abertura) AS primeira_os,
        MAX(o.data_abertura) AS ultima_os
FROM    dbo.auditor_fiscal AS au
        /* (4) junção para trazer o supervisor - qual tipo preserva a matrícula
           1001, que está no topo da hierarquia? */ dbo.auditor_fiscal AS sup
        ON sup.matricula = au.matricula_supervisor
        /* (5) junção com ordem_servico, preservando auditores sem ordem */
        ON o.matricula_auditor = au.matricula
        /* (6) junção com auto_infracao, preservando ordens sem auto */
        ON a.id_os = o.id_os
GROUP BY /* (7) */
ORDER BY autos_lavrados DESC, os_designadas DESC;

```

Conferência.

Verificação	Esperado
Linhas do relatório	20
<code>SUM(os_designadas)</code> sobre todas as linhas	150
<code>SUM(autos_lavrados)</code> sobre todas as linhas	72
Auditores com <code>os_designadas = 0</code>	devem aparecer com zero, nunca ausentes

Análise. (a) Troque a lacuna (1) por `COUNT(*)` e execute. Qual passa a ser a soma da coluna, e por que ela deixa de bater com 150? (b) Retire o `DISTINCT` da lacuna (1), mantendo `COUNT(o.id_os)`. A soma muda? Explique o que a junção com `auto_infracao` fez com as linhas. (c) Sem o `COALESCE` da lacuna (3), o que aparece na coluna de crédito para um auditor sem autos, e por que isso é inaceitável num relatório gerencial?

Atividade 4 — Consistência entre cabeçalho e itens

Regra. *A soma dos valores dos itens de uma NF-e deve ser igual ao valor total do cabeçalho, com tolerância de R\$ 0,01. O relatório deve apresentar, por contribuinte, o total emitido em documentos e o total emitido em itens, e destacar os documentos internamente inconsistentes.*

Fundamento. É verificação de consistência interna: independe de qualquer declaração do contribuinte. É o cenário em que o *fan-out* aparece — somar uma coluna do cabeçalho depois de juntar com a tabela de itens multiplica o valor pelo número de itens do documento.

Agregação exigida. Duas agregações em granularidades distintas — documento e item — consolidadas **antes** da junção, e só então reunidas.

Rascunho — parte A (a demonstração do erro).

```
/* ATIVIDADE 4A - Consulta ERRADA: execute e anote os três números */
SELECT SUM(n.valor_total) AS total_inflacionado,
       COUNT(*) AS linhas_apos_juncao,
       COUNT(DISTINCT n.id_nfe) AS documentos_distintos
FROM   dbo.nfe AS n
       INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
WHERE  n.situacao = 'AUTORIZADA';

/* Agora o total correto, sem a junção: */
SELECT SUM(n.valor_total) AS total_correto, COUNT(*) AS documentos
FROM   dbo.nfe AS n
WHERE  n.situacao = 'AUTORIZADA';
```

Rascunho — parte B (o relatório correto).

```
/* ATIVIDADE 4B - Painel por contribuinte, sem fan-out */
WITH doc AS (
    SELECT n.id_emitente,
           COUNT(*) AS qtd_documentos,
           SUM(n.valor_total) AS valor_documentos
    FROM   dbo.nfe AS n
    WHERE  /* (1) saídas autorizadas */
    GROUP BY n.id_emitente
),
item AS (
    SELECT n.id_emitente,
           COUNT(*) AS qtd_itens,
           SUM(/* (2) coluna de valor do item */) AS valor_itens,
           COUNT(DISTINCT i.ncm) AS ncm_distintos
    FROM   dbo.nfe AS n
           INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
    WHERE  /* (3) o mesmo filtro da CTE anterior */
    GROUP BY n.id_emitente
)
SELECT c.cnpj, c.razao_social,
       COALESCE(d.qtd_documentos, 0) AS qtd_documentos,
       COALESCE(d.valor_documentos, 0) AS valor_documentos,
       COALESCE(it.qtd_itens, 0) AS qtd_itens,
       COALESCE(it.valor_itens, 0) AS valor_itens,
       /* (4) média de itens por documento - cuidado com divisão inteira
          e com denominador zero */ AS itens_por_documento,
       /* (5) diferença entre o total dos cabeçalhos e o total dos itens */
       AS diferenca
FROM   dbo.contribuinte AS c
       /* (6) qual junção preserva os 60 contribuintes, inclusive os que
          nunca emitiram? */ doc AS d ON d.id_emitente = c.id_contribuinte
       /* (7) idem */ item AS it ON it.id_emitente = c.id_contribuinte
ORDER BY ABS(COALESCE(d.valor_documentos,0) - COALESCE(it.valor_itens,0)) DESC;
```

Rascunho — parte C (os documentos inconsistentes).

```

/* ATIVIDADE 4C - Notas em que a soma dos itens diverge do cabeçalho */
WITH itens AS (
    SELECT i.id_nfe, SUM(i.valor_total_item) AS total_itens, COUNT(*) AS qtd_itens
    FROM   dbo.nfe_item AS i
    GROUP BY i.id_nfe
)
SELECT c.cnpj, c.razao_social, n.chave_acesso, n.data_emissao,
       n.valor_total AS valor_cabecalho, t.total_itens, t.qtd_itens,
       n.valor_total - t.total_itens AS diferenca
FROM   dbo.nfe AS n
       INNER JOIN itens AS t ON t.id_nfe = n.id_nfe
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
WHERE  /* (8) o predicado da inconsistência, com tolerância de R$ 0,01 */
ORDER BY ABS(n.valor_total - t.total_itens) DESC;

```

Conferência.

Verificação	Esperado
linhas_apos_juncao em 4A	maior que documentos_distintos — anote a razão
total_inflacionado ÷ total_correto	aproximadamente igual à razão acima
Linhas do painel 4B	60
Documentos internamente inconsistentes (4C)	15

Análise. (a) A razão entre o total inflacionado e o total correto é praticamente idêntica à razão entre linhas e documentos distintos. Explique por quê. (b) Por que `COUNT(DISTINCT n.id_nfe)` devolve o número certo na consulta 4A enquanto `SUM(n.valor_total)` não devolve? (c) Repita a checagem de granularidade para o par `efd_c100 × efd_c170` e registre os dois números.

Atividade 5 — Conciliação NF-e × EFD por período, com subtotais

Regra. Para cada contribuinte do regime **NORMAL** em situação **ATIVO**, o valor total das NF-e de saída autorizadas em cada período de apuração deve coincidir com o valor total dos registros C100 regulares de saída escriturados no mesmo período. O relatório deve trazer subtotal por contribuinte e total geral, e apontar os períodos sem nenhuma escrituração.

Fundamento. É a conciliação de obrigação principal contra obrigação acessória, no nível do período — o formato em que o resultado da malha fiscal chega ao contribuinte na notificação. O período sem escrituração alguma é a omissão de entrega; o período com escrituração de valor menor é a omissão de receita.

Agregação exigida. Agregação nos dois lados **antes** da junção (as duas tabelas são 1:N em relação ao período), junção externa para preservar os períodos sem par, **COALESCE** para transformar ausência em zero, e **ROLLUP** para os subtotais.

Rascunho — parte A (a conciliação).

```

/* ATIVIDADE 5A - Conciliação por contribuinte e período ----- */
WITH emitidas AS (
    SELECT n.id_emitente AS id_contribuinte,
           /* (1) período no formato AAAAMM, para casar com efd_c100.periodo_apuracao
              dica: CONVERT(CHAR(6), n.data_emissao, 112) */ AS periodo,
           COUNT(*) AS qtd_nfe,
           SUM(n.valor_total) AS valor_nfe,
           SUM(n.valor_icms) AS icms_nfe
    FROM   dbo.nfe AS n
    WHERE  n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
    GROUP BY n.id_emitente, /* (2) repita a expressão da lacuna (1) */
),
escrituradas AS (
    SELECT e.id_contribuinte,
           e.periodo_apuracao AS periodo,
           COUNT(*) AS qtd_efd,
           SUM(e.valor_documento) AS valor_efd,
           SUM(e.valor_icms) AS icms_efd
    FROM   dbo.efd_c100 AS e

```

```

WHERE /*(3) apenas documentos regulares de saída */
GROUP BY e.id_contribuinte, e.periodo_apuracao
)
SELECT c.cnpj,
       c.razao_social,
       em.periodo,
       em.qtd_nfe,
       em.valor_nfe,
       COALESCE(es.qtd_efd, 0) AS qtd_efd,
       COALESCE(es.valor_efd, 0) AS valor_efd,
       em.valor_nfe - COALESCE(es.valor_efd, 0) AS diferenca,
       CASE WHEN /* (4) não houve escrituração alguma no período */
             THEN 'SEM ESCRITURACAO NO PERIODO'
             WHEN ABS(em.valor_nfe - es.valor_efd) <= 0.01
             THEN 'CONCILIADO'
             WHEN em.valor_nfe > es.valor_efd
             THEN 'EFD MENOR QUE NF-e'
             ELSE 'EFD MAIOR QUE NF-e'
       END AS resultado
FROM   emitidas AS em
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = em.id_contribuinte
/* (5) qual junção mantém no relatório os períodos em que houve
emissão mas nenhuma escrituração? */ escrituradas AS es
       ON es.id_contribuinte = em.id_contribuinte
       AND es.periodo = em.periodo
WHERE  c.regime_tributario = 'NORMAL'
       AND c.situacao_cadastral = 'ATIVO'
ORDER BY c.razao_social, em.periodo;

```

Rascunho — parte B (omissos do período 202501).

```

/* ATIVIDADE 5B - Contribuintes sem nenhuma escrituração em 202501 */
SELECT c.cnpj, c.razao_social, COUNT(e.id_c100) AS registros_c100
FROM   dbo.contribuinte AS c
       LEFT JOIN dbo.efd_c100 AS e
             ON e.id_contribuinte = c.id_contribuinte
             AND /* (6) o filtro de período - por que ele fica no ON? */
WHERE  c.situacao_cadastral = 'ATIVO'
       AND c.regime_tributario = 'NORMAL'
GROUP BY c.cnpj, c.razao_social
HAVING /* (7) a condição que caracteriza a omissão de entrega */
ORDER BY c.razao_social;

```

Rascunho — parte C (subtotais por região fiscal).

```

/* ATIVIDADE 5C - Pannel por região fiscal com subtotais */
SELECT CASE WHEN GROUPING(m.regiao_fiscal) = 1 THEN 'TOTAL GERAL'
          ELSE COALESCE(m.regiao_fiscal, 'SEM REGIAO (OUTRA UF)') END AS regiao,
       CASE WHEN GROUPING(c.regime_tributario) = 1 THEN 'TODOS OS REGIMES'
          ELSE c.regime_tributario END AS regime,
       COUNT(DISTINCT c.id_contribuinte) AS contribuintes,
       COUNT(*) AS qtd_notas,
       SUM(n.valor_total) AS valor_emitido,
       SUM(n.valor_icms) AS icms_destacado
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
       INNER JOIN dbo.municipio AS m ON m.id_municipio = c.id_municipio
WHERE  n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
GROUP BY /* (8) a extensão do GROUP BY que gera: uma linha por (região, regime),
um subtotal por região e um total geral */ (m.regiao_fiscal, c.regime_tributario)
ORDER BY GROUPING(m.regiao_fiscal), m.regiao_fiscal,
         GROUPING(c.regime_tributario), c.regime_tributario;

```

Conferência.

Verificação	Esperado
Contribuintes ATIVOS do regime NORMAL sem EFD em 202501 (5B)	8
Linha TOTAL GERAL de 5C: qtd_notas	igual à soma dos subtotais de região
Linha TOTAL GERAL de 5C: contribuintes	menor ou igual a 60, e não igual à soma dos subtotais
Períodos classificados como SEM ESCRITURACAO NO PERIODO em 5A	devem existir e ser coerentes com 5B

Análise. (a) Na conferência acima, por que a contagem de contribuintes do total geral **não** é a soma das contagens por região, enquanto a de notas é? (b) Por que o filtro de período da lacuna (6) precisa ficar no **ON** e não no **WHERE**? Descreva o resultado que a consulta produziria se ele fosse movido. (c) Por que a agregação de **nfe** e a de **efd_c100** precisam ser feitas em CTEs separadas, em vez de juntar as duas tabelas e agregar de uma vez? Relacione a resposta com a Atividade 4.

Ficha de conferência

Atividade	Verificação	Esperado	Obtido
1	Carga efetiva abaixo de 4%	2 (id 5 e 17)	
1	Emitentes com mais de 1.000 notas	3 (id 1, 2 e 3)	
1	Emitentes com mais de 500 notas	6	
2	Cancelamento acima de 15%	2 (id 4 e 12)	
2	qtd_notas em 2B vs. 2C	2B menor	
3	Linhas do painel de auditores	20	
3	Soma de os_designadas	150	
3	Soma de autos_lavrados	72	
4	Razão entre total inflacionado e correto	anotar	
4	Linhas do painel por contribuinte	60	
4	Documentos internamente inconsistentes	15	
5	Omissos de EFD em 202501	8	
5	Total geral × soma dos subtotais (notas)	coincidem	

Critérios de avaliação

Critério	Peso
Correção dos resultados (ficha de conferência)	35%
Escolha correta da função de agregação e do nível de agrupamento	20%
Posicionamento correto dos predicados (ON × WHERE × HAVING)	20%
Ausência de fan-out e tratamento adequado de NULL	15%
Qualidade das respostas de análise	10%

19. Referências

Documentação oficial da Microsoft

- *Aggregate Functions (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/functions/aggregate-functions-transact-sql>
- *SELECT — GROUP BY (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/queries/select-group-by-transact-sql>
- *SELECT — HAVING (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/queries/select-having-transact-sql>
- *SELECT — WHERE (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/queries/select-where-transact-sql>
- *STRING_AGG (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/functions/string-agg-transact-sql>

- *GROUPING e GROUPING_ID (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/functions/grouping-transact-sql>
- *OVER Clause (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/queries/select-over-clause-transact-sql>
- *Query Processing Architecture Guide* — <https://learn.microsoft.com/pt-br/sql/relational-databases/query-processing-architecture-guide>

Bibliografia

- ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados*. 7. ed. São Paulo: Pearson. (Funções de agregação e agrupamento em SQL.)
- SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 7. ed. Rio de Janeiro: LTC. (Agregação, **HAVING** e tratamento de valores nulos.)
- DATE, C. J. *SQL and Relational Theory*. 3. ed. O'Reilly. (Crítica ao comportamento de **NULL** em funções de agregação.)

Módulos relacionados do curso

Módulo	Relação
Junções (JOINS)	Pré-requisito direto: o fan-out e a escolha INNER/LEFT
O Modelo Relacional — Conceitos Fundamentais	Agregação como extensão da álgebra relacional
Restrições e Regras de Integridade	Colunas anuláveis por projeto e seu efeito nas médias
Plano de execução e desempenho	Stream Aggregate × Hash Match (Aggregate) ; índices do script 08

Scripts do curso utilizados

Script	Função
01_criar_banco.sql	Estrutura, restrições e índices mínimos
02_inserir_cadastros.sql	Municípios, contribuintes, auditores, pauta
03_inserir_nfe.sql	NF-e e itens
04_inserir_efd.sql	EFD C100 e C170
05_inserir_fiscalizacao.sql	Ordens de serviço e autos de infração
07_verificacao.sql	Conferência da carga e mapa das inconsistências
08_indices_opcionais.sql	Índices de desempenho para consultas agregadas

*Módulo elaborado para o Curso de Banco de Dados Relacional aplicado à Fiscalização Tributária — SEFAZ-PB. Todos os exemplos foram escritos contra a base **curso_integridade_fiscal** gerada pelos scripts 01 a 05, e os valores de conferência derivam do script **07_verificacao.sql**.*